

Computer Networks

Lecture 30 Transport Layer

Content



- TCP

TCP in Internet Stack

Application
HTTP, SMTP, FTP, SSH

TCP provides a reliable, ordered byte stream between processes.

Transport
TCP / UDP

- Connection-oriented
- Reliable
- Flow control
- Congestion control

Network
IP

Link + Physical
Ethernet, Wi-Fi

TCP Segment Field

Field	Purpose	Used in
<i>Sequence number</i>	Byte number of first data byte in the segment	Ordering, retransmission
<i>Acknowledgment number</i>	Next byte expected by receiver	Cumulative ACKs
<i>Flags: SYN, ACK, FIN, RST</i>	Control connection establishment and termination	Handshake, teardown
<i>Window size</i>	Receiver-advertised buffer capacity	Flow control
<i>Checksum</i>	Detects corruption	Reliability

TCP Connection Management

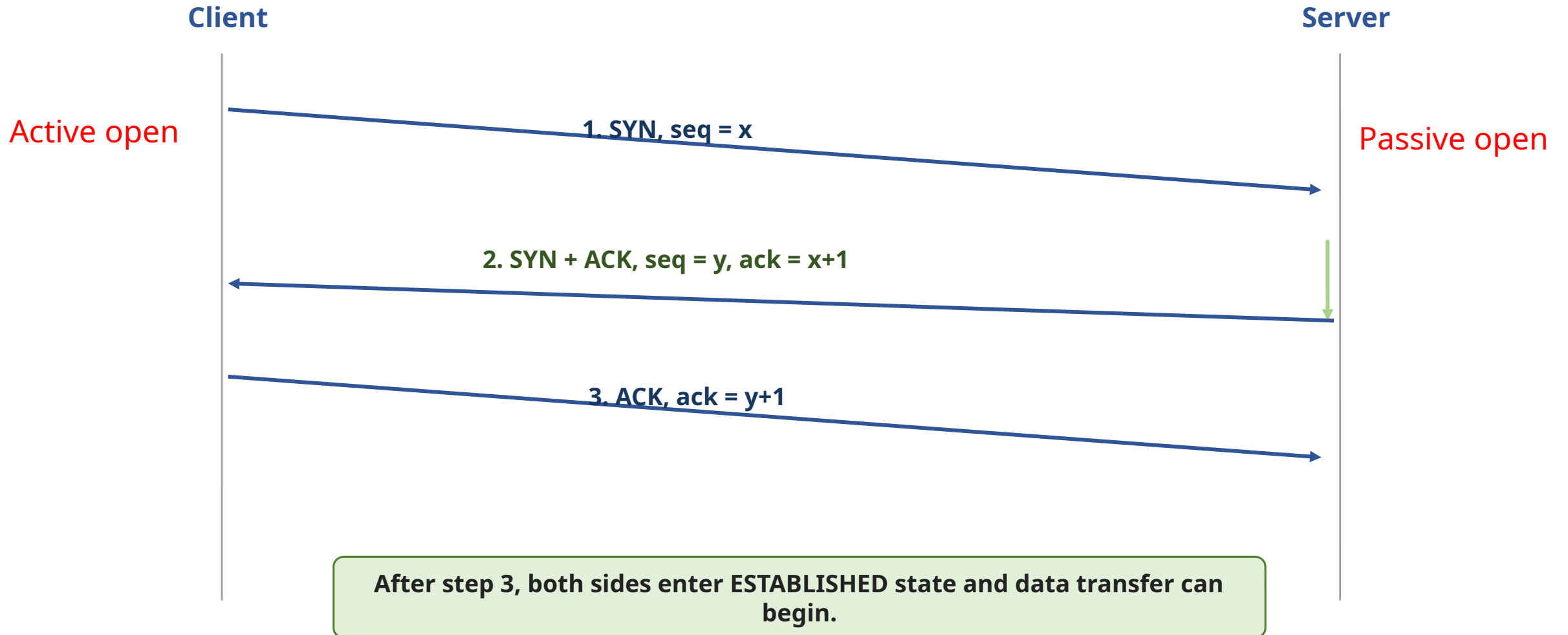
Connection management is not just “starting and stopping.” It is state synchronization between two endpoints.

A connection-oriented transport protocol establishes a virtual path between the source and destination. All the segments belonging to a message are then sent over this virtual path.

In TCP, connection-oriented transmission requires three phases:

- connection establishment
- data transfer, and
- connection termination

TCP Connection Establishment



TCP Connection Establishment

What if both sides try to start the TCP connection at the same time?

In this case both process perform an active open.

So instead of one side waiting and the other side sending SYN, both sides send SYN to each other.

```
Host A ----- SYN -----> Host B
Host A <----- SYN ----- Host B
```

Now both hosts receive a SYN while they have already sent their own SYN.

In response, both send SYN + ACK.

```
Host A ----- SYN + ACK -----> Host B
Host A <----- SYN + ACK ----- Host B
```

Finally, both acknowledge the received SYN + ACK.

```
Host A ----- ACK -----> Host B
Host A <----- ACK ----- Host B
```

After this, only one TCP connection is established, not two.

TCP Connection Establishment

SYN Flooding Attack TCP normally uses a three-way handshake:

```
Client ----- SYN -----> Server
Client <----- SYN + ACK ----- Server
Client ----- ACK -----> Server
```

The problem is that after receiving the first SYN, the server temporarily reserves resources because it expects the client to complete the connection.

TCP Connection Establishment

- In a SYN flooding attack, an attacker sends a very large number of SYN requests to a server.
- But the attacker does something tricky: it uses fake source IP addresses.

Attacker ---- SYN with fake IP 1 ----> Server

Attacker ---- SYN with fake IP 2 ----> Server

Attacker ---- SYN with fake IP 3 ----> Server

Attacker ---- SYN with fake IP 4 ----> Server

The server thinks these are genuine clients trying to connect. So, for each SYN, the server allocates resources

- The server sends SYN + ACK replies to the fake IP addresses.
- But because the IP addresses are fake, the final ACK never comes back.
- So many connections remain **half-open**.
- Eventually the server's resources become full.

Data Transfer

At this stage, both sides can send and receive data.

TCP provides full-duplex communication, meaning data can flow in both directions at the same time.

Host A ----- Data -----> Host B
Host A <----- Data ----- Host B

During data transfer, TCP provides reliable communication using:

- Sequence numbers
- Acknowledgment numbers
- Flow control
- Error control
- Congestion control

Data Transfer: Pushing Data

During the data transfer phase, TCP normally collects data from the application and may wait until enough data is available before sending it efficiently. But sometimes, the application wants the data to be sent immediately, without waiting. This is called pushing data.

Normally, TCP may buffer data.

This improves efficiency because sending many tiny segments can waste network resources.

When the sending application requests a push, TCP sends the data immediately instead of waiting for more data.

Sender Application



Push data now

Sender TCP ----- Data with PSH flag -----> Receiver TCP



Deliver immediately

Receiver Application

Data Transfer: Urgent Data

During the TCP data transfer phase, sometimes an application needs to send some data that should be handled immediately by the receiver. This is called *urgent data*.

Normally, TCP sends data as a continuous byte stream.

Normal data: A B C D E F G H I J

Suppose the sender wants to send an urgent instruction, such as:

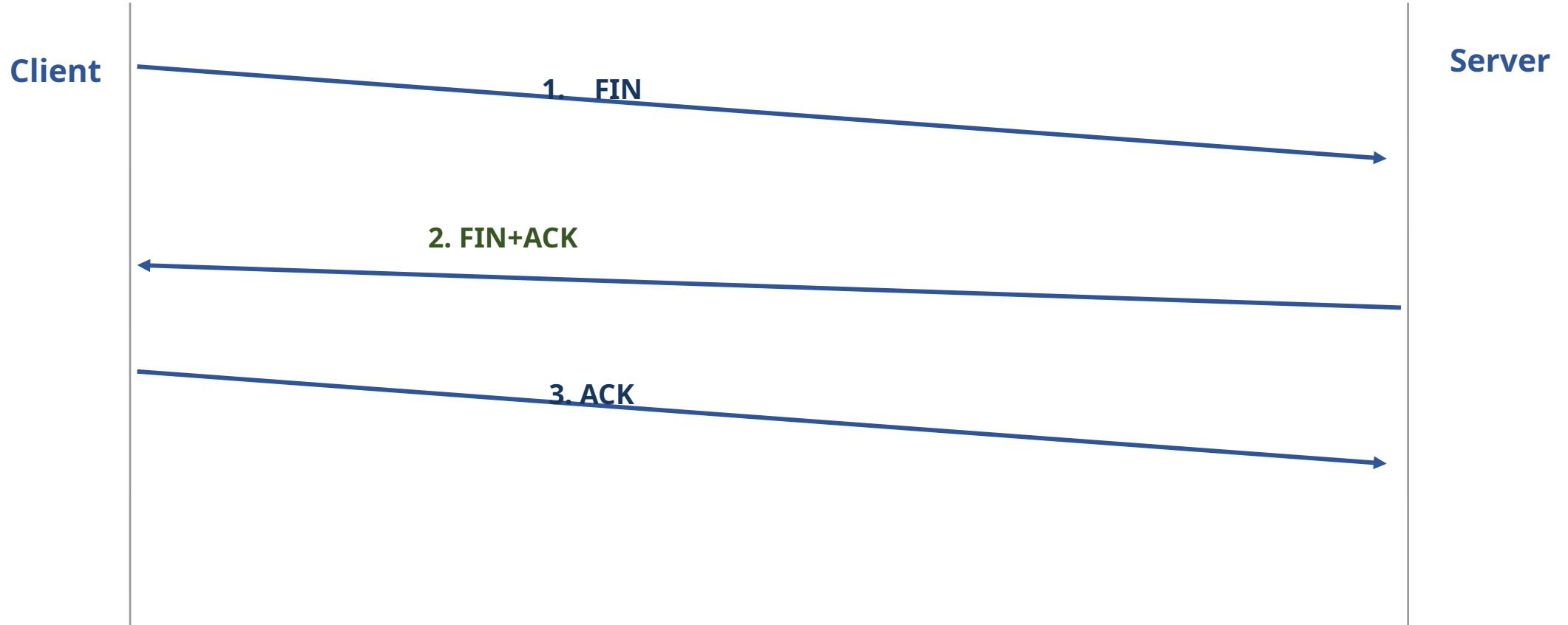
Stop
Interrupt
Cancel
Terminate

Then TCP can mark this part of the data as urgent.

Urgent data in TCP does not create a separate high-priority data channel. Instead, it marks a portion of the byte stream as requiring immediate attention.

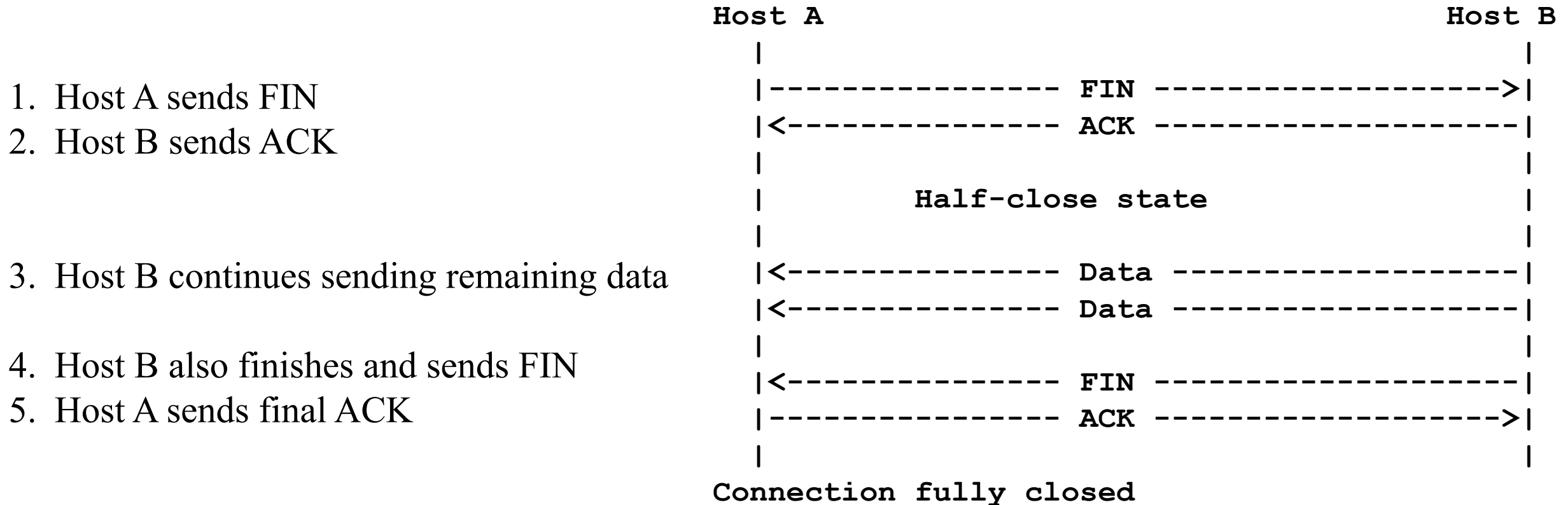
Connection Termination

Three-Way Handshaking



Connection Termination

Half Close: Suppose *Host A* has finished sending data, but *Host B* still has data to send.



TCP Congestion Control

Congestion refers to a network state where the message traffic becomes so heavy that it slows down the network response time.

Problems caused by congestion

- **Packet loss:** Congestion leads to the loss of packets in transit
- **Slow response times:** The Network becomes sluggish and unresponsive
- **Wasted resources:** Retransmissions consume additional bandwidth
- **Poor user experience:** Applications become slow or unresponsive

Congestion control refers to techniques and mechanisms that can either prevent congestion before it happens (proactive approach) or remove congestion after it has happened (reactive approach)

TCP Congestion Control

TCP reacts to congestion by reducing the sender window size. This means sending less data at once to reduce network load.

- **Receiver Window Size:** Indicates the maximum amount of data the receiver can accept without sending an acknowledgement.
- **Congestion Window Size:** Represents the sender's assessment of how much data the network can handle.
- $\text{Sender Window Size} = \text{Minimum}(\text{Receiver Window Size}, \text{Congestion Window Size})$

TCP Congestion Control

Silly Window Syndrome

Silly Window Syndrome is a performance-degrading condition in TCP (Transmission Control Protocol) that occurs when very small data segments (sometimes even smaller than the TCP header of 20 bytes) are transmitted between the sender and receiver.

It can occur due to either:

- Cause 1: The Sender transmits small segments repeatedly
- Cause 2: Receiver advertises a small window repeatedly

TCP Congestion Control

Silly Window Syndrome: Nagle's Algorithm: (Sender-side control)

- ✓ Send the first byte immediately when data is received from the application.
- ✓ Then, buffer all subsequent data in the sender's buffer.
- ✓ Wait until the previous segment is acknowledged (1 RTT).
- ✓ Once the ACK is received, send all buffered data in one segment.
- ✓ Repeat the process.

TCP Congestion Control

Silly Window Syndrome: Clark's Solution: (Receiver-side control)

- ✓ The receiver does not advertise a small window size (e.g., 1 byte).
- ✓ Instead, it waits until a significant portion of the buffer is free.
- ✓ Once enough space is available (e.g., half the buffer or maximum segment size), it sends a larger window update.
- ✓ The sender then sends larger segments to optimize data transmission.

TCP Congestion Control

TCP's general policy for handling congestion consists of the following three phases:

Slow Start

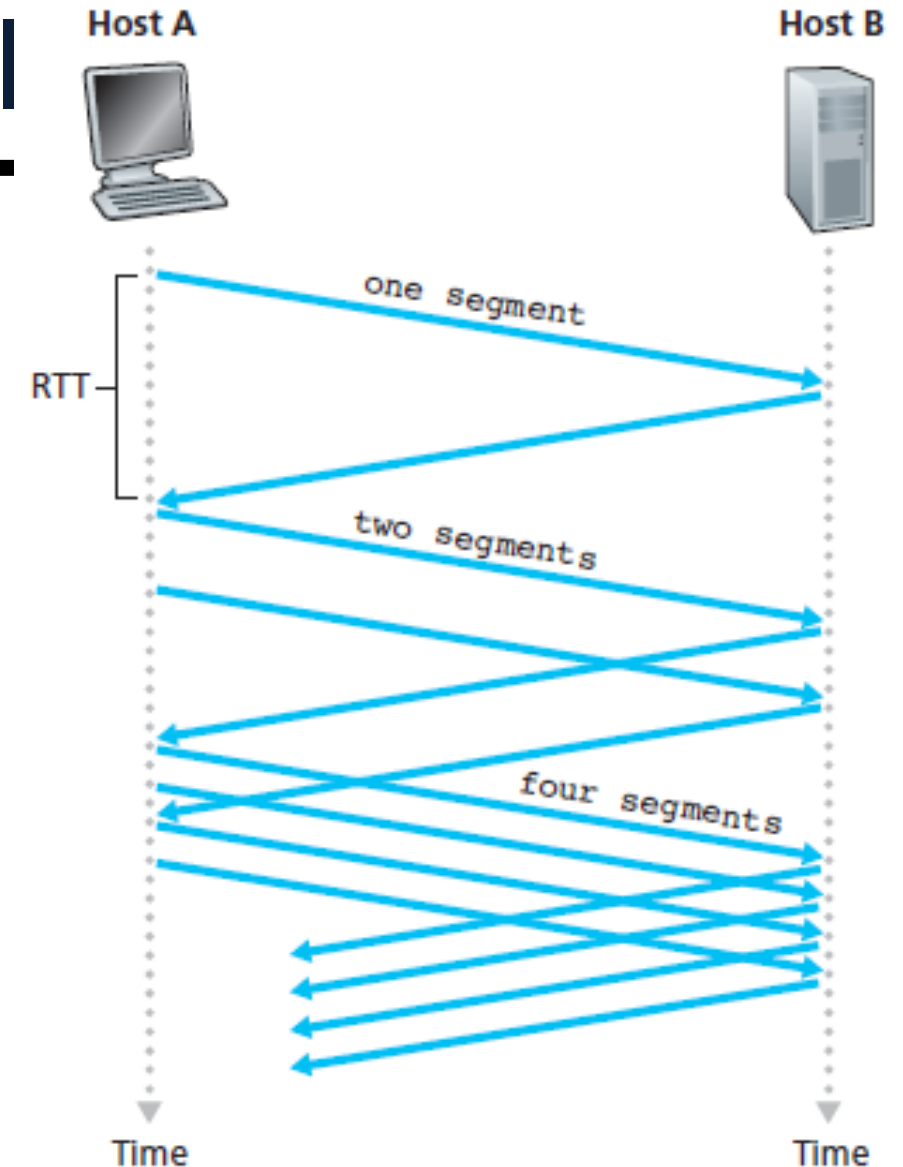
Congestion Avoidance

Congestion Detection

TCP Congestion Control

Slow Start

- Initially, the sender sets the congestion window (CWND) to 1 MSS.
- CWND increases by 1 MSS for each ACK received, effectively doubling every RTT
- The slow start phase continues until the congestion window size reaches the slow start threshold.

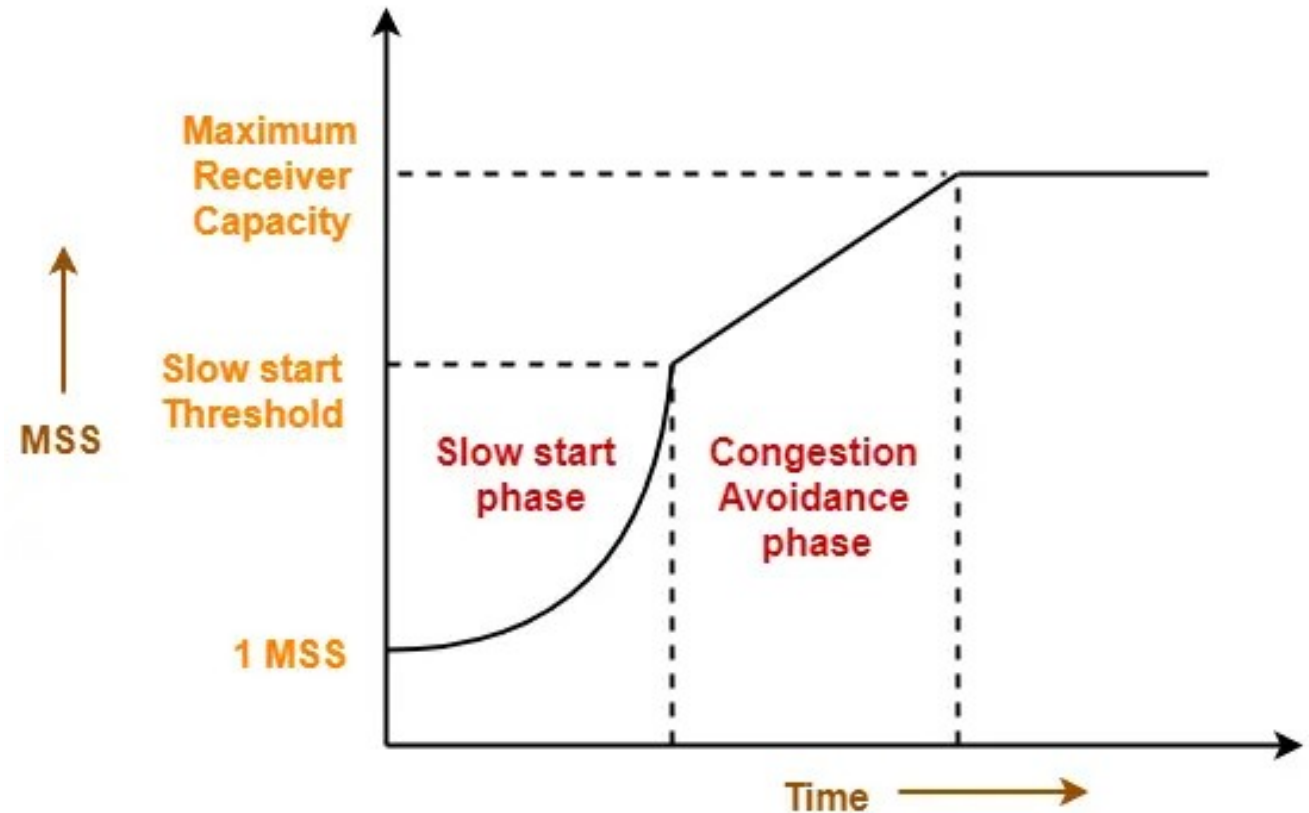


TCP Congestion Control

Congestion Avoidance Phase

After reaching the slow start threshold

- The sender increases the CWND linearly to avoid congestion.
- $CWND = CWND + 1$ (Additive increase) (increase by 1 MSS per RTT)



TCP Congestion Control

Congestion Detection Phase

When the sender detects the loss of segments, it reacts in different ways depending on how the loss is detected:

- Case 1: Detection on time out
- Case 2: Detection on receiving 3 duplicate acknowledgements

TCP Congestion Control

Detection by Timeout

A timeout means the sender sent a segment but did not receive its ACK within the expected time. TCP assumes this is a serious congestion signal.

- `ssthresh = CWND / 2`
- `CWND = 1 MSS`
- Restart slow start
- Retransmit the lost segment

Now TCP starts again from slow start

CWND: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8$

TCP Congestion Control

Detection by 3ACK's

This usually means one segment is lost, but later segments are still reaching the receiver. So TCP assumes the network is still working, but one packet was lost.

- `ssthresh = CWND / 2`
- Retransmit the missing segment immediately
- CWND is reduced, but not usually reset to 1 MSS

TCP Congestion Control

Congestion Control Variants

Over the years, several variants have been developed to optimize performance under different network conditions.

The following are TCP congestion control variants:

- TCP Tahoe
- TCP Reno
- TCP New Reno
- TCP Vegas

TCP Congestion Control

Congestion Control Variants: TCP Tahoe

Case 1: Loss Detected by Timeout

- $ssthresh = CWND / 2$
- $CWND = 1 \text{ MSS}$
- Retransmit lost segment
- Restart slow start

Case 2: Loss Detected by 3 Duplicate ACKs

- $ssthresh = CWND / 2$
- $CWND = 1 \text{ MSS}$
- Retransmit lost segment
- Restart slow start

TCP Congestion Control

Congestion Control Variants: TCP Reno

Case 1: Loss Detected by Timeout

- $ssthresh = CWND / 2$
- $CWND = 1 \text{ MSS}$
- Retransmit lost segment
- Restart slow start

Case 2: Loss Detected by 3 Duplicate ACKs

- Retransmit lost segment without waiting for timeout
- $ssthresh = CWND / 2$
- $CWND = ssthresh + 3 \text{ MSS}$

When the ACK for the retransmitted segment arrives:

- $CWND = ssthresh$
- Enter Congestion Avoidance